

passgen

Детерминированный генератор паролей

Полная документация для защиты проекта
Подробное описание всех функций программы

1. Идея проекта и решаемая проблема

В современном мире у каждого человека есть множество аккаунтов: электронная почта, социальные сети, банки, работа, учёба. Для каждого требуется пароль. Использовать один и тот же пароль везде очень опасно: если злоумышленник узнает пароль от одного сервиса (из-за утечки базы данных), он получит доступ ко всем аккаунтам. Запоминать 20-30 разных сложных паролей человеческая память не позволяет.

Менеджеры паролей (LastPass, Bitwarden) хранят все пароли в одном зашифрованном месте, но это создает единую точку отказа. Если злоумышленник получит доступ к мастер-паролю, все пароли окажутся скомпрометированы.

passgen предлагает принципиально другой подход — детерминированную генерацию. Программа не хранит пароли, а каждый раз вычисляет их заново с помощью криптографического алгоритма HMAC-SHA256. Пользователю нужно запомнить всего одну фразу (сид). Пароль для любого сервиса восстанавливается по формуле: $\text{пароль} = \text{HMAC-SHA256}(\text{сид}, \text{сервис} + \text{длина} + \text{шаг})$.

Преимущества:

- Пароли не нужно хранить — они всегда восстанавливаются из сида и названия сервиса
- Нечего красть — на компьютере нет базы данных с паролями
- Пароль невозможно потерять — при знании сида он восстанавливается в любой момент
- Для каждого сервиса создается уникальный пароль
- Не требуется интернет
- Программа бесплатна и имеет открытый исходный код

2. Структура программы (index.html)

Вся программа находится в одном файле — index.html. Он состоит из трех частей: HTML (разметка страницы), CSS (стили и оформление) и JavaScript (логика работы). Никаких дополнительных файлов, библиотек или серверов не требуется.

2.1. HTML — разметка страницы

Страница состоит из блоков (карточек), расположенных вертикально:

1. Верхняя панель с заголовком "passgen" и ссылками "Экспорт"/"Импорт".
2. Карточка "Сид" — поле для секретной фразы, место для ошибки.
3. Карточка "Сервис" — поле ввода названия, кнопка "список".
4. Карточка "Параметры" — настройки генерации.
5. Кнопка "Создать".

6. Карточка "Логин и пароль" — результат.
7. Кнопка "Копировать".
8. Модальное окно импорта (скрыто).

2.2. CSS — стили и оформление

Цвета задаются через CSS-переменные: светло-фиолетовый фон, белые карточки, темный текст, фиолетовый акцент. Кнопки с градиентом, скругленные углы, тени. Анимации: `fadeUp` (появление), `glow` (ошибка), `slideDown/slideUp` (панели), `checkPop` (кружочки).

2.3. JavaScript — логика программы

Вся логика на JavaScript с использованием Web Crypto API — встроенной в браузер криптографической библиотеки. Реализованы: генерация паролей, сохранение настроек для каждого сервиса, экспорт/импорт JSON, обработка событий.

3. Криптографический алгоритм

3.1. Что такое HMAC-SHA256 и чем отличается от обычного хеша

HMAC-SHA256 — это криптографический алгоритм, который сочетает хеш-функцию SHA-256 с секретным ключом. HMAC расшифровывается как Hash-based Message Authentication Code (код аутентификации сообщения на основе хеша). SHA-256 — это стандартная криптографическая хеш-функция, которая преобразует любые входные данные в 256-битное (32-байтовое) значение фиксированной длины.

Чем HMAC отличается от обычного SHA-256? Обычный SHA-256 просто берет входные данные и вычисляет их хеш. Это работает как отпечаток пальца: по хешу нельзя восстановить исходные данные, но одинаковые данные дают одинаковый хеш. Проблема в том, что любой человек может вычислить SHA-256 от любых данных — для этого не нужен секрет.

HMAC решает эту проблему, добавляя секретный ключ в процесс хеширования. Алгоритм смешивает ключ с данными по специальной схеме (два прохода хеш-функции с разными вариантами ключа). Результат HMAC невозможно вычислить без знания ключа. Даже если злоумышленник узнает результат HMAC, он не сможет восстановить ключ или подделать результат для других данных. В программе `passgen` сид пользователя используется именно как такой секретный ключ.

Простыми словами: если SHA-256 — это "черный ящик", который превращает любую строку в набор символов, то HMAC-SHA256 — это "сейф", который открывается только специальным ключом (сидом). Без ключа вычислить пароль невозможно. С ключом — легко и всегда одинаково для одних и тех же входных данных.

3.2. Пошаговая схема генерации пароля

Шаг 1. Пользователь вводит сид. Программа преобразует фразу в байты через `TextEncoder`. Из байтов создается криптографический ключ через `crypto.subtle.importKey`.

Шаг 2. Формируется сообщение: сервис + длина + номер шага. Номер шага (счетчик) начинается с 0 и увеличивается при необходимости.

Шаг 3. Вычисляется HMAC-SHA256: `crypto.subtle.sign("HMAC", key, message)`. Результат — 32 байта.

Шаг 4. Превращение байтов в символы. Каждый байт проверяется: если меньше порога — превращается в символ из алфавита. Если нет — пропускается. Шаг увеличивается, цикл повторяется до набора нужной длины.

Шаг 5. Инверсия: "Первая заглавная" (`toUpperCase`), "Последняя цифра" (отдельный HMAC).

3.3. Rejection Sampling — обеспечение равномерного распределения

При превращении байтов HMAC в символы возникает математическая проблема. Каждый байт может быть от 0 до 255 (256 значений). Длина алфавита может быть 50 или 62. 256 не делится нацело ни на 50, ни на 62.

Если бы программа просто брала байт % длина_алфавита, некоторые символы встречались бы чаще других. Для 50 символов: байты 0-249 дают каждый символ по 5 раз, но байты 250-255 (6 штук) дают символы 0-5 дополнительно. Это смещение (bias) недопустимо в криптографии.

Решение: $\text{max} = 256 - (256 \% \text{длина_алфавита})$. Байты $\geq \text{max}$ отбрасываются (reject). Используются только байты $< \text{max}$. Поскольку max кратен длине алфавита, распределение символов строго равномерное. Пример: алфавит 50. $\text{max} = 250$. Байты 250-255 отбрасываются. Каждый символ встречается ровно 5 раз.

Этот метод называется Rejection Sampling — стандартная техника в криптографии, используемая в генераторах паролей ведущих менеджеров паролей.

3.4. Алфавиты наборов символов

Заглавные: ABCDEFGHJKLMNPQRSTUVWXYZ (нет I, O — путаются с 1 и 0).

Строчные: abcdefghiklmnpqrstuvwxy (нет j, l, o — путаются с цифрами).

Цифры: 0123456789

Спецсимволы: !@#\$%&*+=-.

По умолчанию: заглавные + строчные + спецсимволы, длина 10.

3.5. Генерация логина

Логин через SHA-256 (без ключа). Берется сид + "login" + сервис + счетчик. Счетчик растет с каждым вызовом — каждый клик новый логин. Первые 6 байтов хеша превращаются в символы из алфавита "abcdefghijklmnopqrstuvwxyz23456789" (нет i,l,o,0,1).

4. Интерфейс программы

Описание каждого элемента интерфейса.

4.1. Поле "Сид"

Самое важное поле. Сюда вводится секретная фраза — единственное, что нужно запомнить. Без сида невозможно восстановить пароль. При пустом поле — розовая пульсирующая граница (анимация glow) и сообщение "Введите сид". При вводе ошибка исчезает. Enter запускает генерацию.

4.2. Поле "Сервис" и список

Ввод названия сервиса. Кнопка "список" открывает панель с сервисами. Популярные сервисы с цветными кружками-логотипами:

Google (синий G), GitHub (черный GH), VK (синий V), Яндекс (красный Ya), Mail (синий @), iCloud (темно-синий i), Onliner (оранжевый O), Kufar (зеленый K).

Клик по сервису — заполняет поле и восстанавливает настройки. Свой сервис можно ввести вручную — после генерации добавится в список. Правый клик удаляет из списка.

4.3. Параметры генерации

Пресеты: Минимальный (8, буквы+цифры), Стандартный (10, кроме спецсимволов), Максимальный (16, всё). Свой — ручная настройка. Длина 1-99, стрелки. Четыре кружочка — наборы символов. Расширенные: "Первая

заглавная", "Последняя цифра".

4.4. Кнопка "Создать"

Запускает полный цикл: проверка сида и чекбоксов, HMAC-SHA256, инверсия, отображение, копирование, добавление в список, сохранение. Фиолетовый градиент, тень.

4.5. Результат

Логин (только чтение) + "Создать логин". Пароль (только чтение) — моноширинный, жирный, по центру. Индикатор: 3 полосы, цвет от розового до сиреневого, рекомендация.

4.6. Кнопка "Копировать"

Копирует в буфер. Clipboard API, при ошибке exesCommand. "Скопировано!" на 1.5с.

4.7. Экспорт и импорт

Экспорт: JSON со всеми сервисами и настройками.

Импорт: загружает JSON, окно выбора сервисов с чекбоксами.

5. Все функции JavaScript (подробное описание)

Подробное описание каждой функции программы человеческим языком, чтобы можно было устно пересказать на защите проекта.

5.0. Схема вызова функций при генерации пароля

```

Пользователь нажимает "Создать" или Enter
|
v
gen()
|-- проверка сида (пусто -> ошибка, выход)
|-- проверка чекбоксов (нет -> ошибка, выход)
|-- составление алфавита
|-- вызов hmacGen(seed, svc, len, alpha)
|   |-- importKey (сид -> ключ HMAC)
|   |-- цикл: sign (HMAC) -> отбор байтов -> символы
|   |-- возврат пароля
|
|-- инверсия (первая заглавная, последняя цифра)
|-- запись пароля в поле pwd
|-- вызов updateStrength(len, sets)
|   |-- сравнение с порогами (14/4, 10/3)
|   |-- отрисовка полосок и текста
|
|-- вызов сру() [копирование в буфер]
|-- добавление сервиса в svcs (если новый)
|-- вызов renderSvcChips() [перерисовка списка]
|-- вызов saveCfg() [сохранение настроек]
    |-- perSvc[сервис] = {state, inv, len}
  
```

Схема для генерации логина (кнопка "Создать логин"):

```
genLogin()
|-- проверка сида (пусто -> ошибка)
|-- проверка сервиса (пусто -> выход)
|-- loginCounter++
|-- digest(SHA-256, seed+"login"+svc+counter)
|-- 6 байтов -> символы из алфавита логина
|-- запись логина в поле login
```

5.1. Функция `init()` — подготовка программы к работе

Функция запускается автоматически при загрузке страницы. Она подготавливает все элементы интерфейса к работе — устанавливает начальные значения, привязывает обработчики событий к полям и кнопкам.

Сначала `init()` вызывает `loadState()` для очистки данных от предыдущего использования. Затем убирает ошибки с поля сида — снимает розовую подсветку и очищает сообщение.

Поле сида получает обработчик `input`: при вводе ошибка снимается. Поля сида и сервиса получают обработчик `keydown`: `Enter` запускает генерацию. Поле сервиса получает `blur`: при потере фокуса программа запоминает выбранный сервис.

Затем `init()` проверяет, есть ли настройки для текущего сервиса. Если есть — восстанавливает. Если нет — устанавливает стандартные: заглавные+строчные+спецсимволы, длина 10. В конце отрисовывает список сервисов с логотипами.

5.2. Функция loadState() — очистка всех данных

Полностью очищает все данные, чтобы каждый запуск начинался с чистого листа. Ключевая функция для безопасности.

Удаляет данные из localStorage. Обнуляет perSvc. Очищает svcs. Сбрасывает текущий сервис. Устанавливает стандартные настройки, длину 10. Очищает поле сервиса.

Благодаря этой функции при повторном открытии программы не остается никаких следов предыдущего использования.

5.3. Функция `saveCfg()` — сохранение настроек для сервиса

Запоминает текущие настройки (чекбоксы, длину, инверсию) для текущего сервиса. При повторном выборе сервиса настройки восстанавливаются автоматически.

Берет `window.state` (чекбоксы), `window.inv` (инверсию), длину из поля. Сохраняет в `perSvc[сервис]`. Только в оперативной памяти — при закрытии исчезают.

5.4. Функция `applyCfg()` — восстановление настроек сервиса

Применяет сохраненные настройки для выбранного сервиса к интерфейсу. Устанавливает длину, чекбоксы, инверсию. Перерисовывает интерфейс.

5.5. Функция `toggleInv()` — переключение расширенных опций

Переключает "Первую заглавную" (cap) или "Последнюю цифру" (dig) при клике. Меняет 0 на 1 и наоборот в `window.inv`. Обновляет кружок. Не сохраняет.

5.6. Функция `applyPreset()` — применение готового пресета

Минимальный: длина 8, caps+lower+digits. Стандартный: длина 10, caps+lower+digits. Максимальный: длина 16, все 4. "Свой" — без действий. Перерисовывает и сохраняет.

5.7. Функция `renderChk()` — отрисовка чекбоксов

Создает 4 строки с кружками и названиями. Активный — фиолетовый (класс "on"). Клик переключает и перерисовывает.

5.8. Функция `renderInv()` — отрисовка инверсии

Обновляет кружки `cap` и `dig`: добавляет или убирает "on".

5.9. Функция `adj()` — изменение длины

+1/-1 по стрелкам. Границы 1-99.

5.10. Функция toggleSvcPanel() — открытие/закрытие списка

Переключает класс "open". max-height 0 <-> 300px. Стрелка поворачивается.

5.11. Функция `renderSvcChips()` — сборка списка сервисов

Сначала популярные (которых нет в `svcs`), потом все сохраненные из `svcs`. Для каждого вызывает `chip()`.

5.12. Функция `chip()` — создание кнопки сервиса

`item` — объект (популярный) или строка (пользовательский). Цветной кружок с буквой или серый с "?". Клик: заполняет поле, применяет настройки. ПКМ: удаляет.

5.13. Функция gen() — генерация пароля (главная функция)

Запускается при нажатии "Создать" или Enter. 10 шагов:

1. Проверка сида — если пусто: розовая подсветка, "Введите сид", останов.
2. Проверка чекбоксов — если ни один не включен: ошибка, останов.
3. Составление алфавита из выбранных наборов.
4. hmacGen() — асинхронное вычисление HMAC-SHA256.
5. Инверсия: первая заглавная (toUpperCase), последняя цифра (отдельный HMAC).
6. Отображение пароля в поле pwd.
7. updateStrength(len, sets) — индикатор надежности.
8. cpy() — копирование в буфер обмена.
9. Добавление сервиса в svcs, сортировка, renderSvcChips().
10. saveCfg() — сохранение настроек.

5.14. Функция hmacGen() — криптографическое ядро

Генерация пароля через HMAC-SHA256. Асинхронная (не блокирует браузер).

seed -> байты -> ключ importKey. Сообщение: svc+len. max = 256 - (256 % alpha.length) для rejection sampling.

Цикл: пока pwd.length < len:

- добавить счетчик ct к сообщению
- crypto.subtle.sign("HMAC", key, message) -> 32 байта
- для байт < max: символ = alpha[байт % alpha.length]
- ct++

Возвращает пароль.

5.15. Функция genLogin() — генерация логина

6 символов. Проверяет сид и сервис. loginCounter++. SHA-256 от seed+"login"+svc+счетчик. Первые 6 байтов -> символы из "abcdefghijklmnopqrstuvwxyz23456789" (нет i,l,o,0,1). Каждый клик новый.

5.16. Функция `updateStrength()` — индикатор надежности

`len` $\geq$ 14 и `sets` $\geq$ 4: отличный (сиреневый, 3 полосы, "Отличный пароль"). `len` $\geq$ 10 и `sets` $\geq$ 3: средний (малиновый, 2, "Хороший, можно длиннее"). Иначе: слабый (розовый, 1, "Добавьте символов или увеличьте длину").

5.17. Функция `сру()` — копирование в буфер

`navigator.clipboard.writeText()`, ошибка -> `execCommand("copy")`. "Скопировано!" на 1.5 секунды.

5.18. Функция `exportData()` — экспорт в JSON

Собирает `perSvc`, `data` = `{svcs, perSvc}`, `JSON.stringify` с отступами, `Blob`, ссылка, `download` = `"passgen_export.json"`.

5.19. Функция `importData()` — импорт из JSON

FileReader, парсинг, проверка полей `perSvc/svcs`, `importDataCache`, `showImportModal()`.

5.20. Функция `showImportModal()` — окно выбора сервисов

Список сервисов с чекбоксами (`checked=true`). Пусто — "Нет сервисов для импорта".

5.21. Функция `confirmImport()` — подтверждение импорта

Отмеченные -> `perSvc[svc]` = данные, добавление в `svcs`, сортировка, перерисовка. "Импорт завершен!" на 3 секунды.

5.22. Функция `closeImportModal()` — закрытие окна

`remove class "open", importDataCache = null.`

5.23. Функция `toggleAdvanced()` — расширенные настройки

`max-height 0 <-> 120px`, `opacity 0 <-> 1`. Стрелка поворачивается.

5.24. Функция `getCfg()` — получение настроек

Если `perSvc[svc]` нет — создает стандартные. Возвращает настройки.

5.25. Функция onSvcChange() — смена сервиса

curSvc = значение поля, есть настройки -> applyCfg().

5.26. Функция saveState() — заглушка безопасности

Пустая. Ранее сохраняла в localStorage. Для совместимости.

6. Обработка ошибок

1. Пустой сид — розовая подсветка, "Введите сид". При вводе исчезает.
2. Нет набора символов — "Выберите хотя бы один тип символов".
3. Длина вне 1-99 — принудительная граница.
4. Неверный файл импорта — сообщение об ошибке.
5. В файле нет сервисов — "Нет сервисов для импорта".
6. Clipboard недоступен — exesCommand как запасной.

7. Экспорт и импорт (формат JSON)

Пример структуры файла:

```
{
  "svcs": ["google", "github", "vk"],
  "perSvc": {
    "google": {
      "state": {"capitals":1,"lowercase":1,"digits":0,"symbols":1},
      "inv": {"cap":0,"dig":1},
      "len": 16
    }
  }
}
```

svcs — массив сервисов. perSvc — настройки для каждого: state (capitals, lowercase, digits, symbols), inv (cap, dig), len — длина.

8. Безопасность

1. Локальность: всё в браузере, данные не уходят в сеть.
2. Сессионный режим: настройки только в RAM, при перезагрузке всё удаляется.
3. Стандартная криптография: HMAC-SHA256 (RFC 2104), банки, TLS, blockchain.
4. Rejection Sampling: равномерное распределение, без modulo bias.
5. Необратимость: зная пароль, нельзя восстановить сид.
6. Разные пароли: название сервиса входит в HMAC.

Ограничения

Не защищает от кейлоггеров. Не требует мастер-пароля (сид = мастер-пароль). Сид нужно хранить отдельно.

9. Запуск программы

Программа — один файл index.html. Запуск:

Открыть index.html в любом современном браузере (Chrome, Firefox, Edge, Safari, Opera) двойным кликом или через меню (Ctrl+O -> выбрать файл).

Требования:

- Любой современный браузер
- Web Crypto API (встроен во все браузеры)
- Интернет не требуется